

Dry Run Exercises : Solutions

Unit 2 Function Call

Complete the dry run table for each fragment of code given in the questions below. **Give** the value(s) printed to the screen when each is executed.

Follow the module dry run style guide in your answers.

1.

```
public static void add34 ()
{
    int result = add(3, 4);           // L1
    System.out.println("is " + result); // L2

    return;
}

public static int add (int a, int b)           // L3
{
    int res = 0;                               // L4

    System.out.println("Add " + a + " and " + b); // L5
    res = a + b;                               // L6

    return res;                               // L7
}
```

with a test call

```
add34 ();
```

Complete the dry run tables (you may not need all rows):

add34 ()			
LINE	result	add(3, 4) =?	printed

add (3, 4)					
LINE	a	b	res	return res	printed

Write EXACTLY what is printed here

2.

```
public static void add2 ()
{
    int x = 5;                // L1
    int y = 3;                // L2
    int result = add(x, y);    // L3
    System.out.println("is " + result);    // L4

    return;
}

public static int add (int a, int b)                // L5
{
    int res = 0;                // L6

    System.out.println("Add " + a + " and " + b);    // L7
    res = a + b;                // L8

    return res;                // L9
}
```

with a test call

```
add2 ( ) ;
```

Complete the dry run tables (you may not need all rows):

add2 ()					
LINE	x	y	result	add(5, 3) =?	printed

add (5, 3)					
LINE	a	b	res	return res	printed

Write EXACTLY what is printed here

3.

```
public static void add2plus ()
{
    int x = 4;                // L1
    int y = 8;                // L2
    int result = addition(x + 3, y + 2); // L3
    System.out.println("Answer: " + result); // L4

    return;
}

public static int addition (int a, int b) // L5
{
    int res = a + b;          // L6
    return res;               // L7
}
```

with a test call

```
add2plus ();
```

Complete the dry run tables (you may not need all rows):

add2plus ()					
LINE	x	y	result	addition(x+3, y+2) =?	printed

addition (7, 10)				
LINE	a	b	res	return res

Write EXACTLY what is printed here

4.

```
public static int double1 (int number_to_double)      // L1
{
    int doubled = number_to_double * 2;              // L2

    return doubled;                                   // L3
} // END double1

public static void doubleOranges ()
{
    final int N = 7;                                  // L4

    int oranges = double1(N);                          // L5
    System.out.println(oranges);                       // L6

    return;                                           // L7
} // END doubleOranges
```

with a test call

```
doubleOranges ();
```

Complete the dry run tables (you may not need all rows):

doubleOranges ()					
LINE	N	oranges	double1(N)	return	printed

double1 (7)			
LINE	number_to_double	doubled	return doubled

Write EXACTLY what is printed here

5.

```
public static int double2 (int number_to_double)           // L1
{
    return (number_to_double * 2);                         // L2
} // END double2

public static void doublePears ()
{
    final int N_PEARs = 4;                                  // L3

    int pears = double2(N_PEARs);                           // L4
    System.out.println(pears);                              // L5

    return;                                                  // L6
} // END doublePears
```

with a test call

```
doublePears ();
```

Complete the dry run tables (you may not need all rows):

doublePears ()					
LINE	N_PEARs	pears	double2(N_PEARs)	return	printed

double2 (4)		
LINE	number_to_double	return (number_to_double * 2)

Write EXACTLY what is printed here

6.

```
public static int double3 (int number_to_double)      // L1
{
    return (number_to_double * 2);                    // L2
} // END double3
```

```
public static int triple3 (int number_to_triple)      // L3
{
    int result = number_to_triple * 3;                // L4
    return result;                                     // L5
} // END triple3
```

```
public static void x6Bananas ()
{
    final int N_BANANAS = 11;                          // L6

    int bananas = double3(N_BANANAS);                  // L7
    bananas = triple3(bananas);                        // L8

    System.out.println(bananas);                       // L9

    return;                                             // L10
} // END x6Bananas
```

with a test call

```
x6Bananas ()
```

Complete the dry run tables (you may not need all rows):

x6Bananas ()						
LINE	N_BANANAS	bananas	double3(N-BANANAS)	triple3(bananas)	return	printed

triple3 (22)			
LINE	number_to_triple	result	return result

double3 (11)		
LINE	number_to_double	return (number_to_double * 2)

Write EXACTLY what is printed here

--

```

7.
public static int double4 (int number_to_double)           // L1
{
    return (number_to_double * 2);                         // L2
} // END double4

public static int triple4 (int number_to_triple)           // L3
{
    return (number_to_triple * 3);                         // L4
} // END triple4

public static void x6Bananas2 ()
{
    final int N_BANANAS = 11;                             // L5

    int bananas = triple4(N_BANANAS);                      // L6
    bananas = double4(bananas);                            // L7

    System.out.println(bananas);                           // L8

    return;                                                 // L9
} // END x6Bananas2

with a test call
    x6Bananas2 ()

```

Complete the dry run tables (you may not need all rows):

x6Bananas2 ()						
LINE	N_BANANAS	bananas	triple4(N-BANANAS)	double4(bananas)	return	printed

triple4 (11)		
LINE	number_to_triple	return (number_to_triple * 3)

double4 (33)		
LINE	number_to_double	return (number_to_double * 2)

Write EXACTLY what is printed here

--

8.

```
public static int double5 (int number_to_double)           // L1
{
    return (number_to_double * 2);                          // L2
} // END double5

public static void quadruplePears ()
{
    final int N_PEARs = 4;                                   // L3

    int pears = double5(N_PEARs);                             // L4
    pears = double5(pears);                                   // L5

    System.out.println(pears);                                // L6

    return;                                                  // L7
} // END quadruplePears
with a test call
    quadruplePears ()
```

Complete the dry run tables (you may not need all rows):

quadruplePears ()						
LINE	N_PEARs	pears	double5(N-PEARs)	double5(pears)	return	printed

double5 (4)		
LINE	number_to_double	return (number_to_double * 2)

double5 (8)		
LINE	number_to_double	return (number_to_double * 2)

Write EXACTLY what is printed here

9.

```
public static int double6 (int number_to_double)      // L1
{
    return (number_to_double * 2);                    // L2
} // END double6

public static void quadruplePears2 ()
{
    final int N_PEARS = 11;                            // L3

    int pears = double6(double6(N_PEARS));             // L4

    System.out.println(pears);                          // L5

    return;                                             // L6
} // END quadruplePears2
with a test call
    quadruplePears2 ()
```

Complete the dry run tables (you may not need all rows):

quadruplePears2 ()						
LINE	N_PEARs	pears	double6(N-PEARs)	double6(double6(...))	return	printed

double6 (11)		
LINE	number_to_double	return (number_to_double * 2)

double6 (2)		
LINE	number_to_double	return (number_to_double * 2)

Write EXACTLY what is printed here

10.

```
public static int square (int to_square)           // L1
{
    int squared = to_square * to_square;           // L2

    System.out.print(squared);                      // L3

    return squared;                                 // L4
} // END square
```

```
public static void squaresquaredCherries ()
{
    final int N_CHERRIES = 3;                      // L5

    int cherries = square(square(N_CHERRIES));      // L6

    System.out.println(cherries);                  // L7

    return;                                         // L8
} // END squaresquaredCherries
```

with a test call

```
squaresquaredCherries ()
```

Complete the dry run tables (you may not need all rows):

squaresquaredCherries ()						
LINE	N_CHERRIES	cherries	square(N-CHERRIES)	square(square(...))	return	printed

square (3)				
LINE	to_square	squared	return (squared)	printed

square (9)				
LINE	to_square	squared	return (squared)	printed

Write EXACTLY what is printed here